P3835R0
John Spicer <jhs@edg.com>
Ville Voutilainen <ville.voutilainen@gmail.com>
Jose Daniel Garcia Sanchez <josedaniel.garcia@uc3m.es>
Audience: EWG, CWG, LWG
2025-09-03

P3835R0: Contracts make C++ less safe -- full stop!

Introduction

Contracts, as specified in P2900R10, were added to the C++ working draft at the February 2025 meeting.

P2900 says "When used correctly, contract assertions can significantly improve the safety and correctness of software".

P2900 also says "In practice, the choice of semantic will most likely be controlled by a command-line option to the compiler".

The Problem (or at least one of the problems)

Users are likely to want to apply a given semantic to a given component of a program. For example, a library provided by a third party might want to make sure all of their code is always compiled with the quick_enforce semantic.

The contracts feature in the CD does not provide a mechanism to make sure that a consistent semantic is provided for a given component.

This was pointed out in the February meeting of WG21 both my John Spicer (and other authors), and also by Eric Fiselier, who did (most or all) of the clang implementation of the feature.

At that meeting, Eric said that the feature was essentially unusable without some kind of label mechanism to allow semantics to be tied to components.

Eric suggested that a vendors agree on a set of attributes that could be used until a solution in the standard is specified.

Such a vendor agreement has not been pursued and in the absence of such an agreement, Eric has implemented a clang-specific set of attributes and options to control this behavior: USING CLANG CONTRACTS

A trivial example

Consider this example where a library provides a header `z.h` and an implementation (that is built into a binary library) of `l1.cpp`. The library is build with the `quick_enforce` semantic to ensure that any violations of the libraries plain-language contract are detected.

The client code as shown in `c1.cpp` is compiled with the `ignore` semantic.

The programmer has no way of knowing which semantic will actually be applied for any of the calls.

If the compiler choses not to inline the call in `l1.cpp` a version of the function emitted as part of the compilation of `c1.cpp` might be used, in which case the contract violation will not be detected.

```
file: z.h

inline void f(int x) {
        contract_assert(x > 0);
}

file: l1.cpp

#include "z.h"

void g() {
        f(0); // expected to fail contract
}

file: c1.cpp

#include "z.h"

extern void g();

int main() {
        f(1); // expected to pass contract
        g();
}
```

It has been suggested that a user with a sufficiently powerful linker and the knowledge to use it appropriately might be able to work around this issue.

That is not a satisfactory answer for an important saftey issue. Even if a user could manage that for simple cases like the one above, it would not be a reasonable solution when more than two components are involved and/or for a larger number of functions.

When does this problem occur?

The problem occurs in any case in which a header file contains an inline function or a template definition (of any kind) that makes use of any kind of contract assertion.

Do modules help?

This issues appears to be orthogonal to modules. If an attribute-like solution were provided, it would presumably apply to uses of clients of the module. In the absence of an attribute-like solution, an inline functtion or template would presumably be treated simmilarly to such an entity appearing in a header file

Conclusion

Contracts reduce the safety of C++ and should be removed until a version that is strictly an improvement in safety is provided.